COMPETITION RULES

Official Technical Specifications

1. Document Overview

This document is the official technical specification for the KDD Cup 2026 Data Agents competition, detailing the packaging method for participant submissions, Docker container runtime environment, I/O format constraints, hardware resource limits, model invocation methods, and scoring mechanism.

All participating teams must carefully read and strictly comply with all requirements in this document before submitting their solutions. The evaluation system will run containers exactly as described in this document. Any submission that does not comply with the specifications will result in evaluation failure or a score of 0.

> **Note:** If there are any discrepancies between this document and other descriptions on the official website, this document's latest version shall prevail. If you have any questions, please contact the organizing committee on Discord or WeChat community.

2. Data Format Specification

2.1 Complete Directory Structure

During evaluation, the complete directory structure inside the container is as follows. The participant's Agent must traverse all task subdirectories under /input, process them sequentially, and write results to the corresponding /output/task_<id>/ directory.

```
/input/                          # Read-only mount, no writes allowed
└── task_<id>/                     # One subdirectory per task, id is unique
    ├── task.json                  # Task metadata (task_id, difficulty, que
    └── context/                   # Context data required for the task
        ├── csv/                   # Optional: Structured CSV files
        ├── db/                    # Optional: SQLite database files
        ├── json/                  # Optional: Structured JSON files
```

```
/output/                              # Read-write, participants write prediction
└──── task_<id>/                        # Corresponds one-to-one with input task
```

Note:

- The specific subdirectories under context/ are not fixed for each task; different tasks may contain different combinations of data sources. Each task contains at least one of csv/, db/, json/, doc/, and may include knowledge.md.
- /input is mounted read-only; any write operation will cause a runtime error. The /output directory is pre-created by the evaluation system; participants only need to write task_<id>/prediction.csv without creating parent directories.

2.2 task.json Format

The task metadata for each task is stored in task.json, with the following field descriptions:

Field Name	Type	Description
task_id	string	Unique task identifier, consistent with /input/task_<id>/ directory name
difficulty	string	Difficulty level: easy / medium / hard / extreme
question	string	Natural language analysis question describing the data analysis task to be answered

task.json example:

```
{
  "task_id": "task_11",
  "difficulty": "medium",
  "question": "Which product category generated the highest total revenue in
}
```

2.3 context/ Data Sources Description

The data source types under the context/ directory vary by task. The participant's Agent must detect and read the subdirectories and files that actually exist:

csv/	Structured Tables	One or more CSV files, directly readable with pandas and other tools
db/	SQLite Database	One or more .sqlite / .db files containing multiple relational tables
json/	Structured Data	Semi-structured data files in JSON format
doc/	Data Documentation	Data reports and analysis documents in Markdown or other formats
knowledge.md	Background Knowledge	Background knowledge document related to the task, including business definitions and terminology explanations

Correspondence between difficulty and data sources:

Difficulty	Data Modality	Document Scale	Core Challenge
Easy	CSV/JSON + knowledge.md	Short text	Python code generation and data analysis
Medium	CSV/JSON + db/ + knowledge.md	Medium	Text-to-SQL + multi-source data analysis
Hard	Same as above + doc/ data documents	~10K–128K tokens	Unstructured document reasoning
Extreme	Same as Hard	>128K tokens	Long context engineering and memory management

2.4 Output Format: prediction.csv

For each task, the participant's Agent must write prediction results to /output/task_<id>/prediction.csv. This file is in standard CSV format (UTF-8 encoding) with the following requirements:

- The first row is the column names (header); column names do not participate in scoring and are only for readability
- From the second row onwards are data rows, each corresponding to one result record
- Column order does not affect scoring; scoring is based on unordered matching of column value vectors (see Section 6)
- Numeric data should retain sufficient precision; the evaluation system will normalize values to 2 decimal places (rounding) before comparison

name column are both acceptable; the evaluation system considers both forms correct

prediction.csv example (corresponding question: list total sales by category):

```
category,total_revenue
Electronics,4200000.00
Clothing,1850000.00
Food,930000.00
```

3. Submission Method

This competition uses Docker images as the only form of solution submission. Participants need to package the complete Agent program and its runtime dependencies into a Docker image, export it as a compressed archive, and submit it via email to the organizers. After receiving the image, the evaluation system will uniformly start the container in a controlled environment, inject runtime configurations, execute the Agent, and collect output results—all fully automated without participant intervention.

3.1 Image Naming Convention

To ensure unique traceability of submitted images, participants must strictly follow the format below to name Docker images and compressed archives:

Item	Format & Description
Image Name	`<team_id>:v<N>`
Archive Filename	`<team_id>_v<N>.tar.gz (colon replaced with underscore in filename)`
<team_id>	Unique team identifier assigned by the system after registration (e.g., team0042)
<N>	Submission sequence number, starting from 1 and incrementing (e.g., v1 for 1st submission, v3 for 3rd)

Example (Team ID is team0042, 3rd submission):

- Image name: team0042:v3
- Archive filename: team0042_v3.tar.gz

Note: The image tag and archive filename must completely match the above specifications, and team_id must be exactly consistent with the team ID assigned by the organizers. Submissions with

Team ID Assignment Schedule

The organizers will assign team IDs in order of team registration from April 21-23, 2026 (AoE) and send emails to team leaders.

3.2 Image Content Requirements

Requirement	Specification
Base Image	No restriction
Image Archive Size	The submitted archive file (<team_id>_v<N>.tar.gz) must be no larger than 10 GB; participants should control Docker image size accordingly
Startup Entry	Must set ENTRYPOINT or CMD, directly executable via docker run without additional parameters
Root Privileges	Allowed to run as root user, but modifying /input mount directory is prohibited

3.3 Image Submission Process

Submission via Email

This competition uses Google Drive for image submission. Participants upload the Docker image archive to their own Google Drive, set sharing permissions, and then send the sharing link to the organizers via email.

Step 1: Upload image archive to Google Drive

- Upload <team_id>_v<N>.tar.gz to any location in your Google Drive.
- Right-click the file → "Share" → "Anyone with the link" → Set permission to "Viewer" → Copy link.

Step 2: Send submission email

Send the following information to the organizers' email (email address: kddcup@hkust-gz.edu.cn):

- Email subject: [KDDCup2026 Data Agents] Submission - <team_id> - v<N>
- Team ID (team_id)
- Submission version number (e.g., v3)
- Google Drive sharing link

Email example:

```
Team ID: team0000
Version: v1
Sharing link: https://drive.google.com/file/d/1QTBRom51ejitPLe9Ke_HKi1PZkAyW
```

Submission Requirement	Description
Sharing Permission	Must be set to "Anyone with the link can view"; otherwise, organizers cannot download and the submission is invalid
File Naming	Archive filename must comply with <team_id>_v<N>.tar.gz format and match the version number in the email
Archive Size	The shared image archive must be no larger than 10 GB; oversized submissions may be rejected before evaluation
File Retention	Do not delete or modify the file on Google Drive until receiving the organizers' evaluation completion notification
Link Validity	Ensure the sharing link remains valid throughout the evaluation period; link expiration will void the submission

3.4 Evaluation System Execution Commands

After the image is uploaded, the evaluation system will execute the following steps in sequence:

Step 1: Decompress and load the image

```
docker load -i team0042_v3.tar.gz
```

Step 2: Start container for evaluation (conceptual representation, actual parameters generated by evaluation platform)

```
docker run --rm \
  --network=eval_net \           # Internal network (only access to evaluati
  --cpus=16 \                    # CPU core limit
  --memory=64g \                 # Memory hard limit
  --memory-swap=64g \            # Disable swap space
  -v /eval/data/input:/input:ro \    # All task inputs (read-only)
  -v /eval/<submission_id>/output:/output:rw \  # All task outputs (read-wri
  -v /eval/<submission_id>/logs:/logs:rw \ # Solution runtime log files (rea
  -e MODEL_API_URL=<model_url> \      # Internal LLM model service URL
  -e MODEL_API_KEY=<api_key> \        # Internal LLM model API Key
```

Note: Both /input and /output mount the entire dataset directory (containing all tasks). The container must traverse all task_<id> subdirectories under /input and process them one by one.

3.5 Injected Environment Variables

Environment Variable Name	Description
`MODEL_API_URL`	Internal network deployed Qwen3.5-35B-A3B model service address (compatible with OpenAI Chat Completions API); all model calls must go through this address
`MODEL_API_KEY`	Model service authentication Key (injected by evaluation system; participants do not need to know the specific value in advance)
`MODEL_NAME`	Model name used during evaluation, value is "qwen3.5-35b-a3b"

3.6 Agent Core Logic Example

Core logic that main.py should implement (pseudocode):

```
import os, json
from pathlib import Path

input_root  = Path("/input")
output_root = Path("/output")

for task_dir in sorted(input_root.iterdir()):
    task_meta = json.loads((task_dir / "task.json").read_text())
    task_id   = task_meta["task_id"]
    question  = task_meta["question"]
    context   = task_dir / "context"

    # ... Run Agent, generate DataFrame result ...

    out_dir = output_root / task_id
    out_dir.mkdir(parents=True, exist_ok=True)
    result.to_csv(out_dir / "prediction.csv", index=False)
```

3.7 Runtime Log Specification

Participants must ensure that /logs contains complete logs needed for troubleshooting, especially output when the program crashes. Since stdout/stderr are not automatically persisted after container exit, it is recommended to redirect both standard output and standard error to a log file in the startup command:

```
# Synchronously save stdout/stderr in ENTRYPOINT script or startup command
python main.py 2>&1 | tee /logs/runtime.log
```

Important: The organizers will review /logs contents. Any logs containing leaked test set data, gold answers, or other private information will result in the loss of debug support eligibility for that submission; serious cases will result in disqualification.

4. Hardware Resource Limits

4.1 Resource Quota

Resource Type	Limit Specification
CPU	16 cores (vCPU), x86-64 architecture
Memory	64 GB RAM; container will be OOM killed if exceeded, all results for that submission voided
GPU	Participant containers do not have GPU; model inference is uniformly completed by the evaluation system (see Section 5)
Runtime Limit	Total runtime limit for all tasks is 12 hours; container forcibly terminated after timeout

Note: 12 hours is the total time limit for all tasks, not per task. Please allocate processing time for each task reasonably and implement timeout protection in the code to ensure that results for completed tasks can be written out in time. Additionally, we recommend participants use multi-threading/multi-processing for batch parallel acceleration of different input samples (refer to the max_workers setting in the Starter Kit code).

4.2 Timeout & OOM Handling

- **OOM Kill:** Container is forcibly terminated; already written prediction.csv files participate in scoring; unwritten tasks receive 0 points.
- **Non-zero exit code:** Does not affect scoring; the evaluation system only cares whether files exist in the /output directory.

5. Model Invocation Specification

5.1 Development Stage vs Evaluation Stage

This competition adopts a "develop freely, evaluate uniformly" strategy for model usage:

Stage	Model Source	Description
Development / Local Debugging	Participant's choice (any LLM)	Organizers do not provide development API; participants apply for and use various model services on their own
Official Evaluation (After Submission)	Qwen3.5-35B-A3B (Uniformly Deployed by Organizers)	Evaluation system injects model service address via environment variables when starting container; all participants use the same model to ensure fairness

Key Design Principle: Participant code must read model service address and API Key from environment variables and cannot hardcode them in the image. This way, the same code points to the participant's own model during local development and automatically switches to the organizers' uniformly deployed Qwen3.5-35B-A3B after submission.

Fairness Constraint: During official evaluation, Qwen3.5-35B-A3B is the only allowed primary task-solving LLM. Participants must not run any other LLM inside the container as the main reasoning or generation engine, including CPU-executed local inference of alternative LLMs. Auxiliary models of appropriate size are allowed within the official hardware limits for document processing, retrieval, or representation purposes, such as embedding models, provided they are not used as the primary solver.

5.2 Runtime Environment Variable Injection

When starting participant containers, the evaluation system injects runtime environment variables in the form of docker run -e KEY=VALUE. Participant programs must read these configurations from environment variables at runtime and must not hardcode API URLs, API Keys, and other information in code or images.

`MODEL_API_URL`	Internal network deployed Qwen3.5-35B-A3B model service address (compatible with OpenAI Chat Completions API)
`MODEL_API_KEY`	Model service authentication Key (injected by evaluation system; participants do not need to know the specific value in advance)
`MODEL_NAME`	Model name used during evaluation, value is "qwen3.5-35b-a3b"

⚠ **Warning:** It is strictly prohibited to hardcode API Keys, API URLs, and other sensitive configurations in code, configuration files, or Docker images. Environment variables injected by the evaluation system at startup are the only legitimate configuration source.

5.3 Organizer vLLM Deployment Reference

To help participants align local testing with the official evaluation environment, the organizer-hosted Qwen3.5-35B-A3B service is conceptually deployed with vLLM using parameters equivalent to the following command. Sensitive infrastructure fields are intentionally replaced with placeholders. During evaluation, participants must still use MODEL_API_URL, MODEL_API_KEY, and MODEL_NAME instead of hardcoding endpoint details.

```
vllm serve <model_path> \
  --host <host> \
  --port <port> \
  --tensor-parallel-size 8 \
  --seed 1024 \
  --served-model-name qwen3.5-35b-a3b \
  --max-model-len 262144 \
  --reasoning-parser qwen3 \
  --enable-auto-tool-choice \
  --tool-call-parser qwen3_coder \
  --trust-remote-code
```

Parameter	Participant Interpretation
`<model_path>, --host, --port`	Internal deployment details are intentionally hidden. Do not hardcode them; always read the actual endpoint from runtime environment variables.
`--tensor-parallel-size 8`	The official backend serves the model with 8-way tensor parallelism. This is an infrastructure detail and does not change the API contract seen by participant code.

`--seed 1024`	The service uses a fixed backend seed for reproducibility. Participants should still treat outputs as request-dependent and avoid assuming perfect determinism across different prompts or tool traces.
`--served-model-name qwen3.5-35b-a3b`	This is the evaluation-time model identifier and matches the value injected through MODEL_NAME.
`--max-model-len 262144`	The service exposes a maximum context window of 262,144 tokens. Prompt messages, retrieved context, tool schemas, tool arguments, reasoning-related tokens, and generated outputs all consume this budget.
`--reasoning-parser qwen3`	The backend is configured for Qwen3 reasoning parsing. Participants should program against the compatible API response returned by their chosen SDK or HTTP client rather than depending on parser-specific internal formats.
`--enable-auto-tool-choice`	The official service enables model-side automatic tool selection. Participants may supply OpenAI-compatible tool definitions and let the model choose when to call them.
`--tool-call-parser qwen3_coder`	Tool calling is parsed with the Qwen3 Coder format, which is why tool use should follow the standard compatible schema exposed by vLLM's OpenAI-style API.
`--trust-remote-code`	The model is loaded with vendor-provided remote code enabled. This affects organizer deployment only and is not something participant code should depend on.

Practical implication: During official evaluation, the service supports long-context inference, OpenAI-style Tool Calling, and Qwen3 reasoning parsing. The stable integration contract for submissions remains the runtime environment variables plus the compatible chat completions endpoint exposed through MODEL_API_URL.

5.4 Recommended Code Patterns

Method 1: Using OpenAI SDK (Recommended)

```
import os
from openai import OpenAI

# Read from environment variables—set your own values during local developme
# injected by system during evaluation
```

```
)
model_name = os.environ.get("MODEL_NAME", "qwen3.5-35b-a3b")

response = client.chat.completions.create(
    model=model_name,
    messages=[
        {"role": "system", "content": "You are a data analysis agent."},
        {"role": "user",   "content": question},
    ],
    ...,
)
```

Method 2: Using requests to directly call HTTP interface

```
import os, requests

resp = requests.post(
    os.environ["MODEL_API_URL"] + "/v1/chat/completions",
    headers={"Authorization": f"Bearer {os.environ.get('MODEL_API_KEY', 'EMP
    json={
        "model": os.environ.get("MODEL_NAME", "qwen3.5-35b-a3b"),
        "messages": [{"role": "user", "content": question}],
        "temperature": 0.0,
    },
    ...,
)
answer = resp.json()["choices"][0]["message"]["content"]
```

5.5 Network Access Policy

Access Type	Policy
External Internet	Completely blocked during evaluation; cannot access any public IPs or domain names
Internal Model Service	Allowed, accessed through the address specified by MODEL_API_URL environment variable
Other Internal Services	Blocked; container can only access model service endpoint
Inter-Container Communication	Prohibited; each submission has only one container, running independently throughout

Host Ports	All blocked, no ports exposed externally

⚠ **Warning:** Container network has blocked all external internet access. Any attempt to call external LLM services will fail due to network blocking. Please ensure that model calls in the code completely rely on the MODEL_API_URL environment variable.

6. Scoring Mechanism

6.1 Evaluation Process Description

The evaluation system uses a unified automated evaluation program to perform offline batch scoring of participant submitted prediction results. Participants need to submit result files in the specified format (such as prediction.csv), and the system will compare them against corresponding standard answers for evaluation and generate final scores.

All submissions run in the same evaluation environment, and the evaluation process is deterministic execution to ensure consistency and fairness of evaluation results.

6.2 Column Matching Method

Evaluation uses a column-level content consistency matching method (column-level matching), with the core process as follows:

1. Read prediction.csv and gold.csv, parse columns and data content
2. For each column, sort all cell values in that column to construct a "column signature"
3. Count the number of occurrences of column signatures in prediction and gold respectively
4. Match based on column signatures, calculate prediction's coverage of gold

- Ignore column names, match only based on column content
- Ignore row order (implemented through sorting)
- Support duplicate columns (same column signature needs to match the same number of times)

6.3 Scoring Metric Calculation

Evaluation introduces a light penalty for redundant predictions based on coverage (Recall) to more comprehensively measure prediction result quality.

Definitions:

- **Matched Columns:** Number of columns in prediction that successfully match gold (matched by column signature)
- **Gold Columns:** Total number of columns in gold

First calculate:

```
Recall = Matched Columns / Gold Columns
```

Final score is:

```
Score = Recall - λ · (Extra Columns / Predicted Columns)
```

Where:

- Penalty term λ weight is a preset constant, used to balance the relationship between coverage and redundant predictions
- When prediction results contain many unmatched columns, the score will decrease accordingly
- The lower bound of the final score is 0

6.4 Design Rationale

This scoring method moderately constrains the "redundancy level" of prediction results while maintaining a Recall orientation, with the following design goals:

- ✅ **Encourage complete coverage:** Prioritize evaluating whether all target columns are found
- ✅ **Control redundant output:** Penalize predictions containing many irrelevant columns
- ✅ **Maintain robustness:** Do not rely on column names or order, judge only based on data content

Evaluation not only focuses on "whether all are found" (Recall), but also encourages "as concise and accurate as possible" prediction results.

6.5 Value & Time Normalization Rules

Before constructing column signatures, the evaluation system normalizes cell content to reduce the impact of format differences (such as standardization of numeric and time types).

Type	Normalization Rule
Null Values	Empty string, "null", "none", "nan", "nat", "<na>" (case-insensitive) → unified to empty string ""
Numeric	Parsed with Decimal, rounded to 2 decimal places (ROUND_HALF_UP). E.g., 4200000 → "4200000.00", 0.005 → "0.01"
Date	ISO 8601 format (YYYY-MM-DD). E.g., "2024-3-1" must be "2024-03-01"

DateTime	With timezone: converted to UTC (ending with Z); without timezone: keep original ISO format
String	Remove leading/trailing whitespace and \r\n; rest preserved as-is (case-sensitive)
Name Fields	First name + last name as two columns OR combined as full name in one column both accepted. E.g., "John" + "Smith" or "John Smith"

⚠️ **Warning:** String comparison is case-sensitive. For example, "East Asia" and "east asia" are considered different values. Please ensure prediction output matches the original string format in the data source.

6.6 Scoring Examples

prediction.csv	gold.csv	Result & Reason
Columns A, B, C all exist and values match completely	Columns B, C	✅ High score: covers all gold columns; but slightly below perfect due to extra column A
Column B exists, C missing	Columns B, C	⚠️ Low score: incomplete coverage of gold columns
Columns B, C exist but B values mismatch	Columns B, C	⚠️ Low score: partial column match failure
Contains many irrelevant columns	Columns B, C	⚠️ Significantly lower score: many redundant columns incur penalty
prediction.csv does not exist	Any	❌ Score 0: missing file

6.7 Total Score & Leaderboard

- Each task's score is calculated independently according to the above rules
- Total score is the average of all task scores
- Leaderboard is sorted by total score in descending order
- During Phase 1, the hidden set is split into A-board and B-board subsets with approximately 60 and 320 tasks, respectively. The final Phase 1 leaderboard combines A-board and B-board scores using weights aligned with the data proportions.
- If total scores are the same, sort by last valid submission time in ascending order (earlier submission ranks higher)

To ensure reasonable allocation of evaluation resources, submission frequency is subject to the following rules:

Limit Item	Rule
Submission Prerequisite	Must wait for the previous submission's evaluation results before making the next submission
Evaluation Timeliness	Results are updated according to the current A/B-board evaluation schedule and queue status; during the April 30–May 2 (inclusive, EoD AoE) workflow migration window, the leaderboard will refresh daily at 00:00 AoE for submissions already received
Daily Submission Limit	Each team can submit at most 1 time per day
Phase 1 Total Submissions	Each team can submit at most 30 times during Phase 1
Phase 1 Hidden Set Size	Approximately 60 A-board tasks and 320 B-board tasks; final counts follow the deployed evaluation configuration
A-Board Runtime Limit	Maximum wall-clock runtime for a single A-board evaluation run is 2 hours; other evaluation settings follow the Rules unless separately announced
Submission Freeze	Submission intake is paused from April 30 to May 2, 2026 (inclusive, EoD AoE) while the updated evaluation workflow is migrated; intake reopens May 3, 2026 (AoE); teams without a prior submission may submit once by April 29, 2026 (EoD AoE); existing submissions continue to be evaluated
Final Submission Deadline	May 20, 2026 (EoD AoE)
Final Phase 1 Leaderboard	Released on May 25, 2026 (EoD AoE) after B-board evaluation
Leaderboard Score	Displays highest score
Image Version	Each submission requires uploading complete image package; incremental updates not supported

page. After evaluation is complete, an email notification will be sent, and then you can submit the next version.

- The above next-version rule does not apply during the April 30–May 2 submission freeze window (EoD AoE), when new Docker image submissions are temporarily disabled for workflow migration; intake reopens May 3, 2026 (AoE).
- Depending on evaluation workload, organizers reserve the right to adjust evaluation time.

8. Prohibited Behaviors & Integrity Standards

The following behaviors will result in evaluation failure; serious cases will result in disqualification:

1. Attempting to access external internet during container runtime, or bypassing the evaluation system's injected MODEL_API_URL to call other LLM services.

2. Running any non-Qwen3.5-35B-A3B LLM inside the evaluation container as the primary task-solving model, including CPU-based local inference of alternative LLMs. Auxiliary models such as embedding models are allowed only when used for document processing or retrieval within the official hardware limits and not as the main reasoning or generation engine.

3. Conducting any form of probing, attacks, or container escape on the evaluation system infrastructure.

4. Modifying the /input mount directory inside the container or destroying environment variables injected by the evaluation system.

5. Multiple teams sharing the same Docker image or submitting the same solution under different team names.

6. Obtaining test set answers through other cheating methods and using them in code.

7. Manual intervention in the runtime process (evaluation is fully automated; interactive operations are not allowed).

8. The organizing committee reserves the right to evaluate, analyze, and conduct academic research on submitted solutions. The organizing committee commits that, without explicit authorization from participants, it will not publish participants' specific implementation details as independent technical solutions.

9. Frequently Asked Questions (FAQ)

Q1: Does the container need to traverse all tasks itself?

Yes. One container processes all tasks in the entire dataset. The Agent needs to traverse all task_<id> directories under /input, run inference independently for each task, and write results to the corresponding /output/task_<id>/prediction.csv.

model address and Key from environment variables and cannot be hardcoded. After submitting the Docker image, the evaluation system will inject environment variables pointing to Qwen3.5-35B-A3B, and the code will automatically switch to the organizers' unified model. During official evaluation, no other LLM may be run inside the container as the primary task-solving model, including CPU-based local LLM inference. However, participants may still use appropriately sized auxiliary models, such as embedding models, within the official hardware limits for document processing and retrieval.

Q3: Must prediction.csv column names match gold.csv?

No. Column names do not participate in scoring; the evaluation script only compares each column's value vector (column_values signature). Column names can be named arbitrarily; semantic names are recommended for easier debugging.

Q4: How is numeric precision handled?

The evaluation script normalizes all numeric values to 2 decimal places (rounding). For example, predicted value 4200000 and gold value 4200000.00 are considered the same. It is recommended to use sufficient precision (at least 2 decimal places) when predicting.

Q5: Is the 12-hour timeout for a single task or all tasks?

12 hours is the total time limit for the entire container runtime, including processing time for all tasks. It is recommended to set separate timeout control for each task in the code and write out <task_id>/prediction.csv immediately after processing each task to prevent timeout from causing loss of completed results.

Q6: Are data files under context/ fixed?

Not fixed. Different tasks may contain different combinations of data sources (csv/, db/, json/, doc/, knowledge.md). The Agent should dynamically detect subdirectories and files that actually exist under context/ rather than assuming a fixed structure.

Committee

FAQ

CONTACT

For questions about the competition, please reach out to the primary contacts.

Boyan Li

Yuyu Luo

COMMUNITY

Discord — KDD Cup 2026 | DataAgents

WeChat Official Account: 数据智能与分析实验室 DIAL

Reply `KDD进群` for the WeChat group QR code, or `KDD大赛` for competition FAQ and resources.